

# **MKID-IFTS Instrument Simulator**

## **Technical Development Guide**

Module Specifications, Data Flow, and Implementation Reference

March 2026 | Python 3.10+

# 1. Architecture

## 1.1 Package Layout

```

mkid_ifts_sim/
  __init__.py
  config.py           # Central configuration dataclass
  source.py           # M1: Astrophysical source spectra
  atmosphere.py       # M2: Atmospheric transmission
  sky_background.py   # M3: Sky and thermal background
  telescope.py        # M4: Telescope collecting area and
reflectivity
  ifts.py             # M5: IFTS optics, interferogram generation
mkid_detector.py      # M6: MKID QE, energy resolution, noise
order_sorting.py      # M7: Order separation strategies
spectrum_recovery.py  # M8: FFT, phase correction, apodization,
stitching
  snr.py              # M9: SNR calculation and noise budget
  etc.py              # M10: Exposure Time Calculator
  plotting.py         # Visualization utilities
  data/               # Bundled curves, sky models, templates
  tests/              # Unit and integration tests

```

## 1.2 Central Configuration (config.py)

Every module receives parameters through a single `InstrumentConfig` dataclass. This ensures consistency across the pipeline and makes parameter sweeps trivial.

```

@dataclass
class InstrumentConfig:
    # Telescope
    D_primary: float      # Primary mirror diameter [m]
    obscuration: float     # Central obscuration ratio [0-1]
    altitude_m: float      # Observatory altitude [m]
    n_reflections: int     # Number of mirror bounces
    mirror_coating: str    # 'aluminum' | 'silver'
    # IFTS
    delta_x_m: float       # Mirror step size [m]
    n_steps: int           # Total mirror steps
    zpd_fraction: float    # Fraction of steps before ZPD
    opd_jitter_rms_m: float # Mirror position error RMS [m]
    dual_output: bool      # Two complementary output ports
    # MKID
    R_energy_ref: float    # Energy resolving power at  $\lambda_{\text{ref}}$ 
    R_energy_ref_nm: float # Reference wavelength [nm]
    R_energy_scaling: str  # 'sqrt' | 'linear' | 'flat'

```

```

qe_model: str          # QE curve identifier
dead_time_us: float    # Per-photon dead time [μs]
max_count_rate_hz: float # Max count rate per pixel [Hz]
dark_rate_hz: float    # Dark count rate [Hz/pixel]
# Observation
t_exp_per_step_s: float # Exposure time per step [s]
n_pixels: int          # Spatial pixels to bin
airmass: float         # Airmass
moon_phase: str        # 'new'|'crescent'|'quarter'|'full'
use_nd_filter: bool    # Neutral density filter on/off
nd_transmission: float # ND filter transmission [0-1]
# Order separation
strategy: str          # 'hard_cut' | 'probabilistic'
k_sigma: float         # Grey-zone half-width (Strategy 1)
# Processing
apodization: str       # 'none'|'norton-beer'|'gaussian'|'hanning'
phase_method: str      # 'mertz' | 'forman'

```

### 1.3 Data Flow

The pipeline flows strictly through the modules in dependency order. All internal calculations use wavenumber  $\sigma = 1/\lambda$  [cm<sup>-1</sup>] as the native spectral coordinate, since the FTS naturally operates in wavenumber space. Convert to wavelength only for user-facing output.

```

Source (M1) → Atmosphere (M2) → +Sky (M3) → Telescope (M4) → IFTS (M5)
           → MKID (M6) → OrderSort (M7) → FFT+Stitch (M8) → SNR (M9) → ETC (M10)

```

**Important:** Sky background must be added BEFORE interferogram generation (M5), not after. The sky is modulated by the interferometer identically to the source signal. This is what causes the distributed noise property of an FTS.

### 1.4 Two Execution Modes

**Full simulation mode:** Modules M1–M8 execute sequentially, generating a realistic interferogram with noise and recovering the spectrum via FFT. Used for validation and strategy comparison.

**Analytical mode:** Modules M1–M4 compute the photon rates, then M9 computes SNR directly from closed-form noise expressions (bypassing M5–M8). Used by the ETC (M10). Must agree with full simulation to within 10%.

## 2. Module Specifications

### M1: Source Spectra (source.py)

**Input:** Source type, normalization (magnitude or surface brightness), redshift, line widths.

**Output:** Spectrum object: wavelength array  $\lambda$  [nm] on a uniform-in-wavenumber grid (9091–28571  $\text{cm}^{-1}$ ), flux  $F(\lambda)$  [photons/s/cm<sup>2</sup>/nm].

**Templates:** Flat continuum, Planck blackbody (variable T), HII region (H $\alpha$ , [NII], [SII], [OIII], [OII], H $\beta$  with adjustable ratios), planetary nebula, stellar library (O–M types), composite galaxy. Store as CSV in data/templates/.

#### Key functions

```
load_template(name, **kwargs) → Spectrum
blackbody(T_eff, mag, band) → Spectrum
emission_lines(lines_dict, fwhm_kms) → Spectrum
from_file(filepath) → Spectrum
apply_redshift(spectrum, z) → Spectrum
```

#### User interface note

The ETC must accept input in formats astronomers use: magnitude + spectral type for point sources, surface brightness + template for extended sources, or a user-supplied spectrum file (FITS or CSV). The source module must handle all three entry points.

### M2: Atmospheric Transmission (atmosphere.py)

**Input:** Site altitude, airmass, optional PWV.

**Output:**  $T_{\text{atm}}(\lambda)$  array, values [0, 1]. Multiplicative on source flux.

**Physics:**  $T(\lambda, X) \approx T(\lambda, 1)^X$  where X is airmass. Key features: Rayleigh scattering ( $\propto 1/\lambda^4$ ), O<sub>2</sub> A-band at 760 nm, H<sub>2</sub>O bands at 720/820/940 nm.

**Data source:** Pre-tabulated from ESO SkyCalc or Gemini for the target site. One table at airmass 1.0; scale via exponent.

#### Key functions

```
transmission(wavelength_nm, airmass, altitude_m) → ndarray
```

### M3: Sky Background (sky\_background.py)

**Input:** Moon phase, site altitude, pixel scale.

**Output:**  $B_{\text{sky}}(\lambda)$  in [photons/s/cm<sup>2</sup>/nm/arcsec<sup>2</sup>]. Additive to source before interferogram generation.

**Components:** (a) Moonlight + zodiacal continuum scaled by phase; (b) OH airglow lines (Rousselot et al. 2000 line list), dominant at 600–1100 nm; (c) atomic airglow: [OI] 557.7 nm, [OI] 630.0 nm, Na D 589 nm; (d) thermal continuum from telescope/dome above 950 nm.

### Key functions

```
sky_spectrum(wavelength_nm, moon_phase, altitude_m) → ndarray
oh_lines(wavelength_nm) → ndarray
thermal_background(wavelength_nm, T_amb, emissivity) → ndarray
```

## M4: Telescope Throughput (telescope.py)

**Input:** Source spectrum (post-atmosphere), sky spectrum, config.

**Output:**  $N_{\text{in}}(\lambda)$  [photons/s/nm] entering instrument.

**Computation:**  $N_{\text{in}}(\lambda) = [S_{\text{atm}}(\lambda) + B_{\text{sky}}(\lambda) \times \Omega_{\text{pix}}] \times A_{\text{eff}} \times R_{\text{mirror}}(\lambda)^n$ . Aluminum reflectivity: ~92% at 500–1100 nm, ~88% at 400 nm, ~82% at 350 nm, with aging parameter.

### Key functions

```
collecting_area(D_m, obscuration) → float # cm²
mirror_reflectivity(wavelength_nm, coating, n_refl) → ndarray
throughput(wavelength_nm, config, source_flux, sky_flux) → ndarray
```

## M5: IFTS Optics and Interferogram (ifts.py)

**Purpose:** Core physics module. Generates the raw interferogram, the actual data product of the instrument.

### Sub-components

- **T\_optics( $\lambda$ ):** Collimator + camera transmission. Typically 80–95%, mildly wavelength-dependent.
- **ME( $\lambda$ ):** Beamsplitter modulation efficiency =  $4R(\lambda)T(\lambda)$ . Perfect 50/50  $\rightarrow$  ME = 1; 60/40  $\rightarrow$  ME = 0.96.
- **Interferogram equation:**  $I_j(x_k) = 0.5 \int B(\sigma) T_{\text{optics}} ME [1 + (-1)^j \cos(2\pi\sigma x_k)] d\sigma$ , where  $j = 0, 1$  for two ports,  $x_k = (k - k_{\text{ZPD}})\Delta x + \delta x_k$  (Gaussian jitter).
- **Dual-output combination:**  $(I_0 - I_1)/(I_0 + I_1)$  cancels the DC component (broadband flux offset), removes atmospheric extinction variations, and yields the pure modulated signal with  $\sqrt{2}$  noise improvement.
- **Photon noise:** Poisson noise applied per step:  $N_{\text{detected}} \sim \text{Poisson}(N_{\text{expected}} \times t_{\text{exp}})$ .

### Numerical implementation

Compute as matrix multiplication  $I = M \cdot B$ , where  $M$  is the ( $n_{\text{steps}} \times n_{\text{wavenumber}}$ ) modulation matrix with entries  $\cos(2\pi\sigma_i x_k) \times ME(\sigma_i)$ . Avoids loops; executes in milliseconds.

### Folding order geometry

Step size  $\Delta x$  sets Nyquist wavenumber  $\sigma_{\text{Nyq}} = 1/(2\Delta x)$  and free spectral range  $\text{FSR} = \sigma_{\text{Nyq}}$ . Full range 9091–28571  $\text{cm}^{-1}$  folds into  $N_{\text{orders}} = \text{ceil}((\sigma_{\text{max}} - \sigma_{\text{min}})/\text{FSR})$  orders, each spanning  $[\sigma_{\text{Nyq}} \times n, \sigma_{\text{Nyq}} \times (n+1)]$ . Choose step size so order boundaries avoid key spectral features.

### Key functions

```

opd_positions(config) → ndarray # [m]
modulation_matrix(sigma_cm, opd_m, me) → ndarray
generate_interferogram(spectrum, sigma, config) → ndarray
add_photon_noise(interferogram, t_exp) → ndarray
combine_dual_output(I0, I1) → ndarray
folding_orders(config) → OrderLayout

```

## M6: MKID Detector (mkid\_detector.py)

**Purpose:** Model per-photon detection, energy measurement, and detector-level effects.

### Energy resolution model

Each photon with true energy  $E = hc/\lambda$  produces  $E_{\text{meas}} \sim N(E, \sigma_E)$  where  $\sigma_E = E/R_E(\lambda)$ . Resolving power scales with photon energy because quasiparticle number  $N_{\text{qp}} \propto E/\Delta$  (de Visser thesis):

$$R_E(\lambda) = R_{E_{\text{ref}}} \times \sqrt{\lambda_{\text{ref}} / \lambda} \quad \# \text{ 'sqrt' mode}$$

This is the Fano-limited (optimistic) regime. The config supports 'linear' and 'flat' modes to explore degraded detector performance and additional noise contributions from readout power.

### Saturation and count-rate effects

- **Dead time:**  $N_{\text{detected}} = N_{\text{true}} / (1 + N_{\text{true}} \times \tau_{\text{dead}})$ . Causes nonlinearity at high flux.
- **Maximum count rate:** If the predicted per-pixel rate exceeds `max_count_rate_hz`, the ETC issues a saturation warning and suggests enabling the ND filter.
- **Dark counts:** Default zero. MKIDs at  $T < 100$  mK have exponentially suppressed thermal quasiparticles.

### Key functions

```
qe(wavelength_nm, model) → ndarray
energy_resolution(wavelength_nm, config) → ndarray # R_E(λ)
energy_sigma_eV(wavelength_nm, config) → ndarray
apply_dead_time(rate_hz, dead_time_us) → ndarray
check_saturation(rate_hz, config) → bool
```

## M7: Order Separation (order\_sorting.py)

**Purpose:** The novel research module. Implements two strategies for splitting the broadband data into per-order interferograms using the MKID energy tag on each photon.

**Key distinction from CCD-based FTS:** Because MKIDs are photon-counting detectors, order separation happens per photon BEFORE the interferogram is assembled. Each photon is individually tagged with a measured energy and assigned (or weighted across) orders. The per-order interferograms are then built from the assigned photons. This is fundamentally different from a CCD-based FTS, where the interferogram is an integrated intensity and order separation must be done optically.

### Strategy 1: Hard-cut with grey-zone exclusion

If  $|E_{\text{meas}} - E_{\text{boundary}}| > k\sigma_E$  for all order boundaries, assign to most probable order. Otherwise discard. Parameter  $k_{\text{sigma}}$  (default 2.0) controls the trade-off.

```
hard_cut_assignment(E_meas, sigma_E, order_boundaries,
                    k_sigma) → ndarray # order index; -1 = discarded
```

### Strategy 2: Probabilistic weighting

Compute  $P(\text{order}_n | E_{\text{meas}})$  from Gaussian overlap with each order's energy range. Each photon contributes fractionally to multiple orders' interferograms.

```
probabilistic_assignment(E_meas, sigma_E,
                         order_boundaries) → ndarray # (n_phot, n_ord)
```

### Analytical fast model (for ETC)

Closed-form expressions for contamination fraction (Strategy 2) and grey-zone photon loss (Strategy 1). Must match Monte Carlo to within 5%.

```
contamination_fraction(R_E, order_width_eV) → float
grey_zone_loss(R_E, order_width_eV, k_sigma) → float
```

### Output

Array of shape  $(n_{\text{orders}}, n_{\text{steps}})$ : one interferogram per folding order, ready for FFT.

## M8: Spectrum Recovery (spectrum\_recovery.py)

**Purpose:** Transform each order's interferogram to recover the spectrum, then stitch orders into a broadband result.

### Processing steps

1. **DC removal:** If dual-output combination was not performed, subtract the mean of each order's interferogram to remove the broadband offset. With dual-output  $(I_0 - I_1)/(I_0 + I_1)$ , the DC is already cancelled.



2. **Phase correction:** Compute phase from the double-sided portion (steps before ZPD). Apply Mertz or Forman method to produce a symmetric interferogram.
3. **Apodization (optional):** Window functions: none (sinc ILS, max resolution), Norton-Beer (medium), Gaussian, Hanning. Trades resolution ( $\sim 1.2\text{--}2\times$  broadening) for sidelobe suppression.
4. **FFT:** `numpy.fft.rfft` per order. Output grid spacing:  $\Delta\sigma = 1/(2 \times n\_steps \times \Delta x) \text{ cm}^{-1}$ .
5. **Order stitching:** Concatenate per-order spectra. At order boundaries, the edges of each order's FFT have lower SNR (fewer photons contribute near band edges). Apply a weighting function that tapers each order at its edges and blends with the adjacent order in the overlap region. Without this, visible artifacts appear at order boundaries.

### Key functions

```
remove_dc(interferogram) → ndarray
phase_correction(interferogram, n_zpd, method) → ndarray
apodize(interferogram, window) → ndarray
fft_to_spectrum(interferogram, delta_x) → (sigma, spectrum)
stitch_orders(order_spectra, order_layout,
              taper_width_bins=10) → Spectrum
```

## M9: Signal-to-Noise Ratio (`snr.py`)

**Purpose:** Compute SNR per spectral resolution element. Both full (post-FFT) and analytical modes.

### Noise budget

$$\text{SNR}(\sigma) = S(\sigma) / \sqrt{S(\sigma) + N_{\text{sky\_dist}}(\sigma) + N_{\text{contam}}(\sigma)^2}$$

- **S( $\sigma$ ):** Source photons per spectral channel after all throughput.
- **N<sub>sky\_dist</sub>:** Distributed sky noise. In an FTS, sky photon noise from the entire bandpass distributes across all channels. Per-channel contribution  $\approx \sqrt{N_{\text{sky\_total\_per\_step}}} / \sqrt{N_{\text{channels\_in\_order}}}$ . With MKID order-sorting, only sky within the order contributes, which is the key advantage over a filter-free CCD-FTS.
- **N<sub>contam</sub>:** Order cross-contamination from M7. Strategy- and R<sub>E</sub>-dependent.
- **Spatial binning:** SNR improves by  $\sqrt{n_{\text{pixels}}}$ .

### Key functions

```
compute_snr(source_spec, sky_spec, config, strategy) → SNRResult
noise_breakdown(config, source_spec, sky_spec) → NoiseComponents
```

## M10: Exposure Time Calculator (etc.py)

**Purpose:** Astronomer-facing tool. Uses analytical mode (M9) for speed.

### Operating modes

- **Mode A:** Given exposure time  $\rightarrow$  return SNR( $\lambda$ ) curve.
- **Mode B:** Given target SNR at reference  $\lambda \rightarrow$  return exposure time [s].
- **Mode C:** Given science goal  $\rightarrow$  return recommended config + exposure time.

### Additional checks

- **Saturation warning:** If predicted count rate exceeds `max_count_rate_hz` on any pixel, warn the user and suggest ND filter or shorter per-step exposure.
- **Performance bound:** Must execute in  $< 1$  second. Validate against full simulation to  $< 10\%$  SNR error.

### User interface

Minimum viable interface: command-line with YAML or JSON config files. Optional future extension: Jupyter notebook widget or web interface. Input must accept magnitude + spectral type, surface brightness + template, or user-supplied spectrum file.

### Key functions

```
snr_from_time(source, config, t_total_s)  $\rightarrow$  SNRResult
time_from_snr(source, config, target_snr, ref_nm)  $\rightarrow$  float
optimize_config(source, science_goal)  $\rightarrow$  InstrumentConfig
```

## 3. Validation and Testing

### 3.1 Unit tests

- **M1:** Blackbody peak matches Wien's law; flux conserved under redshift.
- **M2:**  $T_{\text{atm}} = 1$  at airmass  $\rightarrow 0$ ;  $\text{O}_2$  absorption present at 760 nm.
- **M5:** Monochromatic source  $\rightarrow$  pure cosine interferogram; dual-output cancels DC.
- **M6:**  $R_E(350 \text{ nm}) > R_E(1100 \text{ nm})$ ; dead-time model linear at low rate; saturation flag triggers at `max_count_rate`.
- **M7:** At  $R_E \rightarrow \infty$ : contamination  $\rightarrow 0$ , grey-zone loss  $\rightarrow 0$ . Analytical matches Monte Carlo within 5%.
- **M8:** Round-trip: input spectrum  $\rightarrow$  interferogram  $\rightarrow$  FFT = input (noiseless). DC removal produces zero-mean interferogram. Stitched spectrum has no visible artifacts at order boundaries.

### 3.2 Integration tests

- Monochromatic source through full pipeline  $\rightarrow$  sinc at correct  $\sigma$  with correct amplitude.
- Zero sky background  $\rightarrow$   $\text{SNR} = \sqrt{N_{\text{source}}}$  per spectral channel.
- SITELLE-equivalent config (SN3 filter,  $\Delta x = 2943 \text{ nm}$ , no MKID sorting)  $\rightarrow$  compare to SITELLE ETC.
- Analytical mode SNR vs. full simulation SNR: agreement within 10% across all test cases.

## 4. Development Phases

Phases are ordered by dependency: each phase requires the previous one to be complete.

### Phase 1: Foundations

**Modules:** config.py, source.py, atmosphere.py, telescope.py.

**Milestone:** Can generate an input photon rate  $N_{\text{in}}(\lambda)$  for any source, atmosphere, and telescope combination. All four modules have passing unit tests.

### Phase 2: FTS Engine

**Modules:** ifts.py, spectrum\_recovery.py (FFT and phase correction only, no stitching yet).

**Milestone:** Round-trip validation passes: input spectrum  $\rightarrow$  interferogram  $\rightarrow$  FFT = input. Poisson noise added. Dual-output combination working. DC removal verified. Apodization options functional.

### Phase 3: MKID and Order Sorting

**Modules:** mkid\_detector.py, order\_sorting.py, spectrum\_recovery.py (add stitching).

**Milestone:** Both strategies implemented and tested. Analytical contamination/loss models match Monte Carlo. First end-to-end broadband simulation with MKID order sorting runs successfully. Order stitching produces clean broadband spectrum.

### Phase 4: SNR, ETC, and Investigations

**Modules:** snr.py, etc.py, plotting.py.

**Milestone:** ETC runs in  $< 1$  second and agrees with full simulation. Strategy comparison complete for at least three source types. Configuration optimization maps produced. Demonstration notebooks written.

### Phase 5: Completeness

**Modules:** sky\_background.py (full OH line list and moon model). Package documentation.

**Milestone:** Full sky model integrated. SITELLE comparison complete. Package installable via pip. README and API docs written. Publication-ready figures generated.

## 5. References

### 5.1 Code repos

- **MKID-models** ([github.com/sahderooij/MKID-models](https://github.com/sahderooij/MKID-models)): Superconductor physics, quasiparticle dynamics, R\_E from first principles.
- **MKIDPipeline** ([github.com/MazinLab/MKIDPipeline](https://github.com/MazinLab/MKIDPipeline)): Real MKID photon-table handling and wavelength calibration.
- **TiEMPO/DESHIMA** ([github.com/deshima-dev/tiempo\\_deshima](https://github.com/deshima-dev/tiempo_deshima)): End-to-end MKID spectrometer simulator, architectural reference.
- **SITELLE ETC** ([cfht.hawaii.edu](http://cfht.hawaii.edu)): Validation baseline for FTS SNR calculations.

### 5.2 Key papers

- de Visser, P.J. (2014). PhD thesis: quasiparticle dynamics in aluminium superconducting microresonators.
- Drissen, L. et al. (2016). SITELLE: A Primer. FTS principles, noise, modulation efficiency, apodization.
- Rousselot, P. et al. (2000). Night-sky spectral atlas of OH emission lines (A&A 354, 1134).
- Norton, R.H. & Beer, R. (1976). Apodizing functions for Fourier spectrometry.
- Kaplan, S.B. et al. (1976). Quasiparticle and phonon lifetimes in superconductors.

### 5.3 Python libs

- `numpy`, `scipy`: Core numerics, FFT, interpolation, optimization.
- `matplotlib`: Plotting and visualization.
- `astropy`: Units, FITS I/O, cosmological calculations (recommended).
- `skycalc_ipy`: ESO sky model for atmospheric and sky spectra (optional).
- `h5py`: HDF5 storage for large simulation outputs (optional).